The Inference Compiler: An Automated Pipeline for High-Sparsity Neural Network Deployment

Omar Mahmoud Shaker Qawasmi
Dept. of Artificial Intelligence & Data Engineering
Istanbul Technical University (İTÜ)
Istanbul, Türkiye
qawasmi21@itu.edu.tr

Behçet Uğur Töreyin
Dept. of Artificial Intelligence & Data Engineering
Istanbul Technical University (İTÜ)
Istanbul, Türkiye
toreyin@itu.edu.tr

***Abstract*—Deep neural network deployment on edge devices is limited by computational overhead. We present "The Inference Compiler," an automated pipeline transitioning models from Python frameworks into highly optimized C++ executables. It synthesizes Knowledge Distillation, Taylor Expansion Pruning, and Synchronized Post-Training Quantization. For complex architectures, we introduce a Dependency Tracer to maintain residual network symmetry and a parallelized Sensitivity-Aware Recovery (SAR) mechanism to mitigate pruning shockwaves. Evaluated primarily on MLP and ResNet-110 architectures within a strict 2.0% accuracy degradation limit for pruning (with an additional 0.5% tolerance for quantization), the compiler achieved up to 88.08% global sparsity. Physical benchmarks on an AMD Ryzen 5 CPU demonstrate that our SIMD-unrolled C++ generation yields up to an 89.1x end-to-end speedup over Python environments, significantly reducing inference latency on standard mobile hardware.**

***Index Terms*—Model Compression, Knowledge Distillation, Structured Pruning, Graph Optimization, Taylor Expansion, SIMD Optimization, Parallel Programming.**

I. Introduction

While large parameter counts assist gradient descent during training, they introduce significant redundancies that actively hinder deployment on edge devices with strict power and memory budgets. Reducing inference cost without sacrificing predictive capacity is a critical objective for real-world artificial intelligence.

Current optimization workflows suffer from a severe decoupling between training and deployment. Frameworks like TensorFlow and PyTorch prioritize ease of development over hardware-specific execution efficiency. As the *Lottery Ticket Hypothesis* suggests, deep networks inherently contain sparse sub-networks capable of matching the original dense model's performance [1]. Extracting these sub-networks into a format that a standard CPU can execute efficiently remains a primary challenge.

This paper proposes "The Inference Compiler," a software ecosystem that automates the transition from high-level Python models to independent, branchless C++ binaries. By combining algorithmic compression with compiler-level graph optimizations, we target a 70% reduction in memory footprint and significant latency reductions under strict bounds: a 2.5% maximum total accuracy drop and a 60-minute automated compilation window for selected models.

979-8-3195-1046-4/26/$31.00 ©2026 IEEE

II. Related Work

A. Knowledge Distillation (KD)

Knowledge Distillation [2] enables a compact "Student" model to inherit a "Teacher" model's predictive power via output probability distributions (soft targets). Advancements like the *PURSUhInT* framework [3] emphasize intermediate feature matching, forcing the student to mimic the Teacher's internal convolutional behaviors before destructive pruning.

B. Structured vs. Unstructured Pruning

While unstructured pruning achieves high theoretical compression [1], it causes irregular memory accesses, failing to provide latency improvements on standard CPUs. Structured pruning—the removal of entire filters—preserves dense matrix operations, enabling immediate cache-friendly hardware acceleration. Recent advancements emphasize the automated extraction of structural dependencies across complex networks. Generalizing filter removal to arbitrary topologies requires tracing coupled layers that share channel dimensions, shifting compression from manual layer-by-layer selection to autonomous graph-wide dependency tracking [11].

C. Graph-Level Operator Folding

Beyond simple weight reduction, compiler-level graph optimizations are required to overcome the "memory wall." Methods utilized by BASIRA-LAB [4] demonstrate that Batch Normalization (BN) can be algebraically folded into preceding convolutional weights, eliminating redundant element-wise memory operations during inference.

D. Deep Learning Compilers and Edge Runtimes

Automated compilers like Apache TVM [12] and Google's XLA [13] accelerate edge inference via hardware-specific operator fusions, but inherently rely on virtualization layers or runtime engines that introduce memory overhead. Our framework diverges by executing an atomic operation explosion, completely circumventing standalone runtimes by lowering the graph directly into static, zero-dependency C++ code.

E. Post-Training Quantization (PTQ)

Post-Training Quantization (PTQ) mitigates the memory wall by converting weights into low-bit integers (INT8) [14] without the intense computational budgets of Quantization-Aware Training (QAT). Because standard PTQ often destabilizes residual structures due to cross-layer variance, our pipeline synthesizes synchronized scaling evaluation across linked layers to maintain dimensional parity at branch intersections.

III. System Architecture and Methodology

To conceptualize the pipeline's philosophy, consider the execution of a neural network as traveling between geographic points. Knowledge Distillation is equivalent to discovering a shorter, highly optimized route. Pruning represents the strict discipline of sticking to this route without detours. Finally, Compilation—encompassing quantization and graph optimization—is equivalent to upgrading the means of travel itself to a faster vehicle.

Because discovering the optimal route dictates the absolute limits of subsequent steps, KD's role is foundational. Rather than reinventing it, our pipeline leverages proven external distillation algorithms as a pre-conditioning step to establish the highest possible mathematical baseline before structural reduction begins.

The Inference Compiler utilizes an `OptimizationStrategy` manager to orchestrate a sequential flow (Fig. 1).

A. Phase 1: Multi-Source Distillation

To maximize semantic density, the compiler pre-conditions the model using two parallel distillation engines. **1. Relational KD (RKD):** This module maps the geometric relationships between data points, applying a Huber loss to normalized Euclidean distance matrices (D_T and D_S) [5]. **2. Contrastive Representation Distillation (CRD):** For deep topologies, an external distiller leverages high-speed multinomial negative sampling [6]. Combined with Variational Information Distillation (VID) [7], it aligns high-level spatial feature maps.

B. Phase 2: Topological Dependency Tracing

Compressing residual architectures requires maintaining dimensional skip-connections (`Add` layers). If an algorithm prunes the i-th filter in the main convolutional branch, it must symmetrically prune the i-th filter in the shortcut branch; otherwise, the tensors will crash during element-wise addition.

To prevent this, the compiler deploys a `GraphTracer`. This module acts as a structural blueprint reader. It performs a recursive backward traversal from every `Add` node, identifying upstream layers that dictate channel dimensions. It binds these into immutable "Coupled Groups," ensuring destructive pruning actions are symmetrically applied across linked branches.

C. Phase 3: The Unified Pruning Engine

The pruning engine executes the physical extraction of redundant parameters. It abandons traditional, rigid heuristics in favor of a dynamic, structurally aware approach.

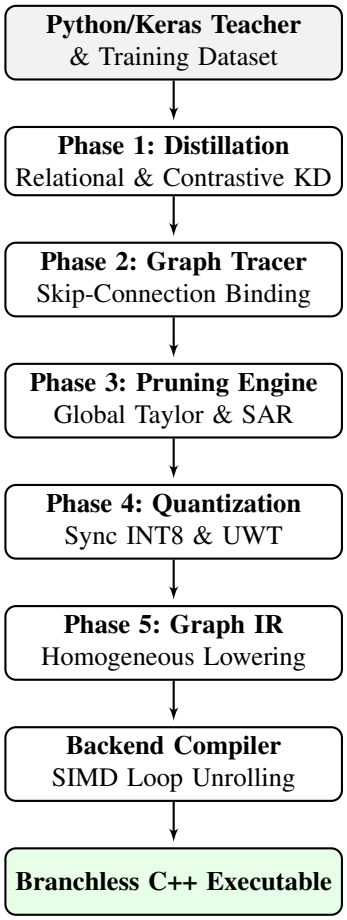

Fig. 1. The Inference Compiler Architecture Pipeline.

1) First-Order Taylor Expansion with Gradient Parallelism: The engine utilizes a Global Taylor Expansion metric [9]. By accumulating the first-order sensitivity of the loss function L with respect to weight W_i, the metric is:

$$\Theta_i = \left| W_i \times \frac{\partial L}{\partial W_i} \right| \tag{1}$$

To mitigate the computational cost of dataset-wide gradient calculation, we implemented **Gradient Parallelism** via compiled tensor-graph functions, executing massively parallel accumulation on the GPU.

2) Sensitivity-Aware Recovery (SAR) & Protection: Standard greedy algorithms fail on dense networks due to "zero-shot entanglement." Our engine slices a dynamic percentage (1.0%-4.0%) of structures and fine-tunes. If accuracy drops $> 0.2\%$, **Sensitivity-Aware Recovery (SAR)** restores weights and tests victims using **Tensor (Mask-Batch) Parallelism** to evaluate perturbed states simultaneously. Load-bearing neurons are placed on cooldown; innocent neurons are safely liquidated.

Deep architectures suffer an "initial shockwave" where SAR prematurely protects nearly all neurons. We engineered a **Concurrent Protection Mechanism** (Algorithm 1) enforcing a 40-cycle grace period to push through this shockwave, with

a $> 90\%$ protection failsafe to prevent infinite loops on exhausted models.

Algorithm 1 Concurrent Protection Mechanism

Input: Network W, Grace $G = 40$, Max Fails $F = 5$
$cycle \leftarrow 0$, $fails \leftarrow 0$
while Pruning Active **do**
  $cycle \leftarrow cycle + 1$
  Prune 1% to 4% batch and Fine-Tune
  if Drop $>$ Threshold **then**
    $prot\_ratio \leftarrow$ SAR_Evaluate($batch$)
    if $cycle \leq G$ **and** $prot\_ratio > 0.90$ **then**
      $fails \leftarrow fails + 1$
      if $fails \geq F$ **then**
        break
      end if
    else if $cycle > G$ **and** $prot\_total > Limit$ **then**
      break
    end if
  else
    $fails \leftarrow 0$
  end if
end while

3) Semi-Structured 1:4 Index Slicing: While hardware relies on NVIDIA's 2:4 sparsity [10], this rigid 50% drop frequently violates our 2.0% accuracy floor. The engine instead ratchets unstructured sparsity by safely slicing the lowest 4.0% of active contiguous 1×4 memory blocks globally per step.

D. Phase 4: Hardware-Aware Quantization

Synchronized Scaling: Standard PTQ breaks residual skip-connections due to varying min/max scalars across parallel branches. The compiler utilizes Tracer metadata to enforce a uniform INT8 scale factor ($S = 127/\max(|W_{group}|)$) across coupled layers [14]. **Unit-Weight Thresholding (UWT):** Weights approximating 1.0 are snapped to exactly 1.0, allowing the C++ generator to bypass redundant multiplications.

IV. Backend Compilation and IR Lowering

A. Atomic Operation Explosion and Homogeneous Lowering

Unlike compilers that treat layers as monolithic blocks [12], [13], our pipeline executes an *Atomic Operation Explosion*, decomposing complex layers into fine-grained arithmetic nodes. This enables Dead Code Elimination (DCE) to physically remove individual Multiply-Accumulate (MAC) instructions associated with pruned 1x4 blocks.

Residual architectures contain isolated Batch Normalization and Activation nodes situated directly at branch intersections. To avoid generating slow, standalone C++ loops that break CPU cache locality, the optimizer executes **Homogeneous Graph Lowering**. Unfusable BN layers are dynamically converted into mathematically equivalent 1×1 convolutions. Standalone Activations are similarly lowered into 1×1 Identity Convolutions parameterized with ReLU triggers.

B. Hardware-Optimized C++ Generation

The backend natively generates branchless C++ code via static memory allocation and **SIMD-Aware Loop Unrolling**. By advancing array pointers symmetrically ($+ = 4$) and physically omitting zero-blocks during code generation, it completely avoids conditional branches inside the core execution loop This optimization directly translates theoretical sparsity into physical hardware acceleration, as demonstrated in Listing 1.

Listing 1. Branchless SIMD MAC Inner Loop

```
#pragma omp parallel for
for(int co=0; co<cout; co+=4) {
    // Zero-blocks omitted to avoid 'if(w!=0)'
        branching
    sum0 += in_val * w_buff[idx];
    sum1 += in_val * w_buff[idx+1];
    sum2 += in_val * w_buff[idx+2];
    sum3 += in_val * w_buff[idx+3];
}
```

V. Experimental Setup and Evaluation

Evaluations adhered to ISO/IEC 25010 performance standards.

Environment Documentation: Physical latency benchmarks were executed natively on an AMD Ryzen 5 5600U processor (16 MB L3 cache). The generated C++ files were compiled via GCC (`-O3 -march=native -ffast-math`) to emulate highly optimized edge deployment profiles. Python/Keras execution represents the unoptimized development baseline.

Constraints: Total accuracy drops were mathematically hard-capped at a 2.0% threshold during the structural pruning phase, with an additional 0.5% tolerance allocated for the post-training quantization bit-shift, establishing a maximum 2.5% degradation ceiling. Operations were bounded by a 60-minute automated compilation window.

VI. Results and Discussion

A. Algorithmic Yield and Sparsity Trade-offs (MLP)

Table I illustrates that applying External KD allowed the MNIST MLP network to tolerate significantly higher structural destruction, achieving an extreme global sparsity of 88.08%. The standard baseline safely achieved 66.80% sparsity with an exact 1.99% drop.

TABLE I
Model Optimization Metrics (MNIST Topology)

Pipeline Strategy	Acc.	Sparsity	Speedup
Baseline (Float32)	98.27%	0.00%	1.00x
Prune + PTQ Only	96.28%	66.80%	3.01x
RKD + Prune + PTQ	94.79%	84.29%	6.35x
Ext-KD + Prune + PTQ	94.70%	**88.08%**	**8.35x**

TABLE II
PHYSICAL LATENCY ON AMD RYZEN 5 5600U (MNIST)

Execution Environment	Sparsity	Latency	Speedup
Python/Keras (Baseline)	0.00%	6.858 ms	1.0x
C++ Gen (Prune Only)	66.80%	0.324 ms	21.2x
C++ Gen (External KD)	88.08%	0.086 ms	79.7x
C++ Gen (Our RKD)	**84.29%**	**0.077 ms**	**89.1x**

B. Physical C++ Latency Verification

Physical latency was benchmarked over 1,000 continuous iterations (Table II).

The RKD pipeline achieved the lowest physical execution time at 0.077 ms (89.1x speedup), outperforming the External KD variant despite a slightly lower theoretical sparsity. RKD inherently preserves filter-wise spatial locality, allowing the CPU's SIMD unit to predictably load contiguous blocks into the L1 cache without fetching "dead" memory, vastly outperforming External KD's fragmented sparsity.

C. ResNet-110 Ablation and Tooling Benchmark

To validate against industry standards and analyze submodule dependencies, we benchmarked the deep ResNet-110 topology against TensorFlow Lite (TFLite) and a structural pruning ablation (Table III). Testing was also conducted on the WideResNet-40-2 teacher model, which successfully achieved 69.41% sparsity while registering a 2.19% accuracy drop, safely operating within the 2.5% combined optimization tolerance.

Standalone isolation benchmarking established a 2.28x relative speedup for TFLite (XNNPACK) over the unoptimized Python baseline. To evaluate our algorithmic contribution, the framework was executed without the Sensitivity-Aware Recovery module (*No SAR Ablation*). Under these conditions, the network hit the degradation limit at only 40.02% sparsity due to early entanglement collapse, reducing accuracy to 67.69%. Conversely, our full pipeline safely bounded residual skip-connections and utilized SAR micro-tuning to heal the network, successfully pushing past the initial shockwave to achieve a highly impressive 65.83% global sparsity while retaining a superior accuracy of 68.17%. This generated an optimized C++ binary that bypassed heavy interpreter runtimes entirely, accelerating execution to 167 ms (a 40.59x speedup over Python).

TABLE III
RESNET-110 ABLATION AND BENCHMARK (CIFAR-100)

Environment	Acc.	Sparsity	Latency	Speed
Python / Keras (Baseline)	69.67%	0.00%	6778 ms	1.00x
TFLite (XNNPACK CPU)	69.67%	0.00%	∼2973 ms	2.28x
C++ (No SAR Ablation)	67.69%	40.02%	452 ms	15.00x
C++ (Our Full RKD)	**68.17%**	**65.83%**	**167 ms**	**40.59x**

VII. CONCLUSION AND FUTURE WORK

The Inference Compiler establishes an automated, mathematically rigorous bridge between the resource-rich training phase and the constrained deployment phase of neural networks. By rejecting rigid threshold heuristics in favor of Gradient-Parallelized Taylor Expansion, the framework safely managed residual connections to achieve 65.83% global sparsity on dense ResNet topologies. Crucially, ablation testing revealed that standard unstructured pruning rapidly collapses; only through the dynamic application of Sensitivity-Aware Recovery (SAR) could the network be healed to reach peak compression limits.

The resulting Intermediate Representation, mathematically lowered into homogeneous linear sequences, allows for the automatic generation of branchless C++ code capable of processing inferences at 0.077 ms on standard mobile CPU hardware, outperforming unoptimized Python baselines by nearly 90x.

Code Availability: The complete source code, deployment scripts, and reproducibility benchmarks for The Inference Compiler are publicly available in our GitHub repository [15].

VIII. ACKNOWLEDGMENTS

The authors would like to acknowledge the use of Bard, for assistance with grammar refinement in the preparation of this manuscript. The content and conclusions presented remain solely the responsibility of the authors.

REFERENCES

[1] J. Frankle and M. Carbin, "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks," *ArXiv*, 2018.
[2] G. Hinton et al., "Distilling the Knowledge in a Neural Network," *ArXiv*, 2015.
[3] R. K. Keser et al., "PURSUhInT: In Search of Informative Hint Points," *ESWA*, 2021.
[4] BASIRA-LAB, "Graph Normalization and Folding Optimizations," 2023.
[5] W. Park, D. Kim, Y. Lu, and M. Cho, "Relational Knowledge Distillation," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
[6] Y. Tian, D. Krishnan, and P. Isola, "Contrastive Representation Distillation," *International Conference on Learning Representations (ICLR)*, 2019.
[7] S. Ahn, S. X. Hu, A. Damianou, N. D. Lawrence, and Z. Dai, "Variational Information Distillation for Knowledge Transfer," *CVPR*, 2019.
[8] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, "FitNets: Hints for Thin Deep Nets," *ICLR*, 2015.
[9] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning Convolutional Neural Networks for Resource Efficient Inference," *ICLR*, 2017.
[10] A. Mishra et al., "Accelerating Sparse Deep Neural Networks," *arXiv preprint arXiv:2104.08378*, 2021.
[11] T. Fang, X. Lu, M. Jiang, and M. Wang, "DepGraph: Towards Any Structural Pruning," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
[12] T. Chen et al., "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.
[13] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," *12th USENIX OSDI*, 2016.
[14] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *CVPR*, 2018.
[15] O. M. S. Qawasmi, GitHub repository, 2026. [Online]. Available: https://github.com/itu-itis-qawasmi21/The-Inference-Compiler